人間のための RuboCop プラグインが AI のハーネスになるまで

Gotanda.rb#70 / Takeshi KOMIYA / 2026-10-28

TIMEINTERMEDIA

自己紹介



- 小宮 健（Takeshi KOMIYA）
  - @tk0miya
- CTO of TIME INTERMEDIA, Inc.
- OSS Developer
  - Sphinx（Python 製ドキュメンテーションツール）
  - rbs_rails メンテナ（Rails アプリ用型生成ツール）
- 書籍: Sphinx をはじめよう（オライリー・ジャパン）

作ってきた RuboCop プラグイン

- rubocop-rbs_inline
- rubocop-rspec-unused-let
- rubocop-numbered-params
- rubocop-prefer_it_parameter
- rubocop-rspec-structure
- rubocop-instance_variable_access
- rubocop-herb

rubocop-rbs_inline

rbs-inline の型注釈の書き方を揃える

```
# bad: 同じ型を二重に書いている
# @rbs name: String
#: (String) -> String
def greet(name) = "Hello, #{name}"
```

rubocop-rbs_inline

rbs-inline の型注釈の書き方を揃える

```
# bad: 同じ型を二重に書いている
# @rbs name: String
#: (String) -> String
def greet(name) = "Hello, #{name}"

# good
# @rbs name: String
# @rbs return: String
def greet(name) = "Hello, #{name}"
```

rubocop-rspec-unused-let

使われていない let を見つける

```ruby
RSpec.describe Foo do
  let(:used)   { 1 }
  let(:unused) { 2 } # 誰も参照していない

  it { expect(used).to eq(1) }
end
```

rubocop-rspec-unused-let

使われていない let を見つける

```ruby
RSpec.describe Foo do
  let(:used)   { 1 }
  let(:unused) { 2 } # 誰も参照していない

  it { expect(used).to eq(1) }
end
```

rubocop-numbered-params

1 行ブロックは numbered parameter で

```
# bad
users.map { |user| user.name }
```

rubocop-numbered-params

1 行ブロックは numbered parameter で

```
# bad
users.map { |user| user.name }

# good
users.map { _1.name }
```

rubocop-prefer_it_parameter

1 行ブロックは it で

```
# bad
users.map { |user| user.name.upcase }
```

rubocop-prefer_it_parameter

1 行ブロックは it で

```
# bad
users.map { |user| user.name.upcase }

# good
users.map { it.name.upcase }
```

rubocop-rspec-structure

context と example を分離する（Jev の試作）

```
# bad
it "when the user is an admin, allows deletion" do
end
```

rubocop-rspec-structure

context と example を分離する（Jev の試作）

```
# bad
it "when the user is an admin, allows deletion" do
end

# good
context "when the user is an admin" do
  it "allows deletion" do
  end
end
```

rubocop-instance_variable_access

インスタンス変数は reader 経由で読む

```
# bad
def full_name = "#{@first_name} #{@last_name}"
```

rubocop-instance_variable_access

インスタンス変数は reader 経由で読む

```ruby
# bad
def full_name = "#{@first_name} #{@last_name}"

# good
attr_reader :first_name, :last_name

def full_name = "#{first_name} #{last_name}"
```

もともとは人間向けだった

- **記法**：新しい記法を広める
  - `_1`、`it`、型注釈
- **ミス**：うっかりミスを見つける
  - 未使用の let 変数
- **規約**：ルールを実行可能にする
  - context 構造、reader 経由のアクセス

AI の進化とともに役割が変わってきた

時代	AI	RuboCop プラグイン
AI が未熟な頃	ミスが多い	人間向けと同じ役割
AI と伴走する時代	精度が上がった	規約を守らせるハーネスに
AI に任せる未来	丸投げ	？

AI が未熟な頃

人間向けの cop と**変わらない**

- **記法**：新しい記法を知らない AI に教える
- **ミス**：多いミスを機械的に正す
- **規約**：ルールを機械的にチェックする

指摘先が人間から AI に変わっただけ

AI と伴走する時代

AI の出力の**精度が上がってきた**

- **記法**：古い書き方を新しい記法に寄せる
- **ミス**：リファクタリング時のゴミを拾う
- **規約**：コードを均質化する
  - 自己検証できる規約 = **ハーネス**
  - プロンプト < RuboCop プラグイン

Pencils Down



https://x.com/dhh/status/2102936073642869121

Pencils Down



https://x.com/nysalor/status/2111091190103966017

AI に任せる未来

- **コードを読まない**時代になるのか？
- それでも**整え続ける意味**はあるのか？

いまの答え

- 答えは不明だが、**整え続けている**
  - しばらくは人間もコードを読み続ける（はず）
  - AI も既存のコードを読んで真似る（はず）
  - 整ったコンパクトなコードには価値がある（はず）
  - 新しい記法に意味があるかは自信なし

最初に紹介した RuboCop プラグインですが…

- rubocop-rbs_inline
- rubocop-rspec-unused-let
- rubocop-numbered-params
- rubocop-prefer_it_parameter
- rubocop-rspec-structure
- rubocop-instance_variable_access
- rubocop-herb

最初に紹介した RuboCop プラグインですが…

- rubocop-rbs_inline
- rubocop-rspec-unused-let
- rubocop-numbered-params
- rubocop-prefer_it_parameter
- rubocop-rspec-structure
- rubocop-instance_variable_access
- **rubocop-herb**
  - 実は今日は触れていませんでした

宣伝

仙台Ruby会議 #03 で rubocop-herb の話をします



宣伝

仙台Ruby会議 #03 で rubocop-herb の話をします



まとめ

- 人間のための RuboCop プラグインは **AI にも効く**
- AI と伴走する今は **AI のハーネス**になった
- AI に任せる未来でも、いまは**整え続ける**
- 仙台Ruby会議 #03、**是非来てください**

ご清聴ありがとう
ございました

